



PURBANCHAL UNIVERSITY
HIMALAYAN WHITEHOUSE INTERNATIONAL COLLEGE
PUTALISADAK, KATHMANDU

A
Project-IV
Report
On

“SEAL: Secure Exam Access & Lockdown”

By

Kabin Bagale [Roll No. 341509]
Ayeal Shakya [Roll No. 341498]
Ashutosh Rupakheti [Roll No. 341486]

Under the Supervision of

Er. Ramesh Sharma

SUBMITTED TO THE DEPARTMENT OF SCIENCE AND TECHNOLOGY
FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF INFORMATION TECHNOLOGY
DEPARTMENT OF SCIENCE AND TECHNOLOGY
Kathmandu, Nepal

October, 2026

CERTIFICATE OF APPROVAL

The Major Project entitled “SEAL: Secure Exam Access & Lockdown”, submitted by Kabin Bagale, Ayeal Shakya, and Ashutosh Rupakheta in partial fulfillment of the requirements for the degree of “Bachelor of Information Technology” has been accepted as a bona fide record of work carried out by them in the department.

Er. Ramesh Sharma

Designation: Supervisor

Er. Bimal Sharma

Head of Department

Department of IT, Computer and Electronics

External Examiner

Purbanchal University

ACKNOWLEDGEMENT

We thank our project supervisor, Er. Ramesh Sharma, for guiding this project from the proposal to the finished system. The comments at the proposal defense asked for a simpler, more honest design and a database that does real work, and SEAL is built around both.

We are grateful to Er. Bimal Sharma, Head of Department, Department of IT, Computer and Electronics, and Er. Aashish Lamsal, Project Coordinator, for the laboratory time, the schedule and the feedback sessions that kept the project on track.

We also thank the faculty members of the Department of IT, Computer and Electronics, Himalayan Whitehouse International College, for their support throughout the semester.

We thank our classmates in the Bachelor of Information Technology programme for their discussions, and our families for giving us the time to finish the work.

Ayeal Shakya

Ashutosh Rupakheti

Kabin Bagale

ABSTRACT

Practical examinations require a system that can manage exam creation, controlled access, timed attempts, answer submission, and grading from a central service. SEAL is a JavaFX desktop application supported by a Spring Boot REST backend and a PostgreSQL database. Administrators create and manage teacher and student accounts, teachers create and publish examinations with multiple choice and written questions, and students access examinations using their registration number, symbol number, and exam access code. The system records examination attempts, validates submitted answers, automatically awards marks for multiple choice questions, and allows teachers to grade written answers manually. Students can view their submitted and graded results through the application. Authentication, role based authorization, password hashing, exam ownership checks, submission deadlines, and database constraints are used to protect the examination process. SEAL currently provides the core examination workflow and grading process while advanced features such as proctoring, operating system lockdown, verified student enrollment, and answer recovery are outside the current implementation.

Keywords: *JavaFX, Spring Boot, REST API, PostgreSQL, Authentication, Role Based Authorization*

CERTIFICATE OF APPROVAL.....	i
ACKNOWLEDGEMENT	ii
ABSTRACT.....	iii
LIST OF FIGURES	vi
LIST OF TABLES.....	vii
LIST OF ABBREVIATIONS.....	viii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives.....	3
1.3.1 General Objective	3
1.3.2 Specific Objectives	3
1.4 Project Scope.....	3
1.5 Report Organization.....	5
CHAPTER 2: LITERATURE OVERVIEW	6
2.1 Background / Theoretical Framework	6
2.2 Study of Related Systems	7
2.3 What's New in Our Project?.....	8
2.4 Functional and Non-Functional Requirements	8
2.5 Feasibility Study	9
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY	11
3.1 Software Development Life Cycle (SDLC).....	11
3.1.1 SDLC Model Used.....	11
3.2 Project Timeline - Gantt Chart.....	12
3.3 System Architecture.....	13
3.4 Data Flow Diagram (DFD)	15
3.4.1 Level 0 - Context Diagram.....	15
3.4.2 Level 1 - Detailed DFD.....	15
3.5 Entity Relationship (ER) Diagram	18
3.6 Use Case Diagram.....	19
3.7 System Flowchart.....	20
3.8 Message Sequence	21
CHAPTER 4: IMPLEMENTATION	23
4.1 Software and Hardware Requirements	23
4.1.1 Technology Stack and Language Used.....	23
4.2 Module-wise Implementation	25
4.2.1 Sign-in and Security Module	25

4.2.2 User and Examination Access Module	26
4.2.3 Examination and Answer Submission Module.....	27
4.2.4 Grading and Results Module	28
4.2.5 Teacher Results and Grading Module.....	28
4.2.6 Student Results and Acknowledgement Module	29
4.2.7 Persistence and Deployment Module.....	29
4.3 Testing.....	30
4.3.3 Test Results Summary	30
CHAPTER 5: FINAL OUTCOME.....	31
5.1 System Output and Analysis	31
5.2 Analysis of Output Against Objectives.....	31
5.3 Final System Screenshots	32
5.4 Performance and Usability Observations	33
CHAPTER 6: CONCLUSION, LIMITATIONS & FUTURE RECOMMENDATIONS	34
6.1 Conclusion	34
6.2 Problems Faced and Limitations.....	34
6.3 Future Recommendations	34
REFERENCES	35
ANNEX.....	36
Annex A: Source Code	36
Annex B: User Manual	36

LIST OF FIGURES

S.No.	Title	Page No.
Figure 3.1	SDLC Model - Iterative and Incremental	12
Figure 3.2	System Architecture Diagram	14
Figure 3.3	Data Flow Diagram - Level 0 (Context Diagram)	15
Figure 3.4	Data Flow Diagram - Level 1	17
Figure 3.5	Entity Relationship (ER) Diagram	18
Figure 3.6	Use Case Diagram	19
Figure 3.7	System Flowchart	20
Figure 3.8	Sequence Diagram - Shell, Console and Database	22
Figure 4.1	Screenshot - Console Sign-in	26
Figure 4.2	Screenshot – Two dashboard student and teacher	27
Figure 4.3	Screenshot – Result and submission panel	27
Figure 4.4	Screenshot – Student registering for exam	28
Figure 5.1	Screenshot – Admin panel demonstrating active and passive users	32

LIST OF TABLES

S.No.	Title	Page No.
Table 2.1	Study of Related Systems	7
Table 2.2	System Requirements	8-9
Table 2.3	Feasibility Analysis	9-10
Table 3.1	Changes Since the Proposal	12-13
Table 3.2	Project Gantt Chart	13
Table 4.1	Hardware and Software Requirements	23
Table 4.2	Syllabus Concepts Used in SEAL	24-25
Table 4.3	Test Cases	29-30

LIST OF ABBREVIATIONS

API	Application Programming Interface
BIT	Bachelor of Information Technology
CSV	Comma-Separated Values
DAO	Data Access Object
DFD	Data Flow Diagram
ER	Entity Relationship
GB	Gigabyte
GUI	Graphical User Interface
HMAC	Hash-based Message Authentication Code
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
JDBC	Java Database Connectivity
JDK	Java Development Kit
JEP	JDK Enhancement Proposal
JSON	JavaScript Object Notation
LAN	Local Area Network
LMS	Learning Management System
PBKDF2	Password-Based Key Derivation Function 2
PC	Personal Computer
RAM	Random Access Memory
SDLC	Software Development Life Cycle
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TC	Test Case
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
USB	Universal Serial Bus

CHAPTER 1: INTRODUCTION

1.1 Background

Practical examinations are an important part of many Bachelor of Information Technology Courses. Subjects such as Java Programming and Database Management Systems require students to work with practical tools, including an integrated development environment, a database client, and a command prompt. The answers are usually source code, SQL queries, or written explanations created on a computer rather than written on paper.

In a traditional computer laboratory examination, supervision is mainly performed by an invigilator who moves between rows of computers. This makes it difficult to maintain consistent supervision across all systems. Collecting answers from individual computers can also be time consuming, and technical problems may interrupt the examination process or cause students to lose unsent work.

Existing examination tools are often designed for online quizzes and restricted browser-based assessments. For example, Safe Exam Browser provides a controlled environment for examinations that can be completed within a browser [1]. This approach is useful for conventional online tests, but it is less suitable for practical examinations where students need to work with development tools and database applications.

SEAL, which stands for Secured Examination Access and Lockdown, addresses the management of practical examinations through a JavaFX desktop application, a Spring Boot REST backend, and a PostgreSQL database. Administrators can manage student and teacher accounts; teachers can create and publish examinations containing multiple choice and written questions, and students can access timed examinations using an exam access code. The system records submissions, automatically grades multiple choice answers, and allows teachers to grade written answers manually. SEAL provides the core structure for

managing practical examinations while leaving advanced features such as operating system lockdown, webcam monitoring, and automated proctoring outside the current scope.

1.2 Problem Statement

Colleges conduct practical examinations in computer laboratories where students must use development tools, database clients, and command line applications. Managing these examinations manually creates several difficulties. Teachers must prepare and distribute examination questions, provide access details, collect completed work, and review student submissions. Without a centralized system, examination records, answer data, and results may be difficult to manage consistently.

Existing examination tools are often designed for browser-based quizzes or restricted online assessments. Lockdown browsers can limit a computer to a controlled environment [1], but this approach is unsuitable for practical examinations that require students to use software such as an IDE, database client, or command prompt. Classroom monitoring tools such as Veyon allow teachers to view student screens [2], but they do not provide examination creation, timed attempts, answer submission, automatic multiple-choice grading, or written answer evaluation. Windows kiosk mode can restrict access to selected applications [3], but it does not manage examination questions, student submissions, or results.

The original proposal included features such as blocking keyboard shortcuts, disabling Task Manager, monitoring running programs, and terminating unauthorized applications. These features require operating system level access, increase implementation complexity, and may create reliability and administrative problems on laboratory computers. They also do not completely prevent students from using external devices or another device. Therefore, the final system focuses on managing the examination process through authentication, role based access, controlled exam access, timed attempts, answer submission, grading, and result management.

SEAL is designed to support the main people involved in the process. Administrators manage teacher and student accounts. Teachers create and publish examinations, define multiple choice and written questions, review submissions, and grade

written answers. Students access published examinations using their credentials and an exam access code, complete the timed examination, submit their answers, and view their results. The system provides a centralized record of examinations, submissions, marks, and account activity within the implemented scope.

1.3 Objectives

1.3.1 General Objective

To design and develop SEAL, a JavaFX based examination system with a Spring Boot REST backend and PostgreSQL database for managing practical examinations, including user authentication, exam creation, timed student attempts, answer submission, grading, and result management.

1.3.2 Specific Objectives

- To design and develop a JavaFX, Spring Boot, and PostgreSQL based examination system with authentication and role based access for administrators, teachers, and students.
- To implement the main examination workflow, including account management, exam creation, exam access, timed attempts, question answering, and answer submission.
- To implement and evaluate result management through automatic multiple choice grading, manual grading of written answers, and result viewing.

Objective 1 is covered in Chapters 1 and 2, objective 2 in Chapter 3, objectives 3.

1.4 Project Scope

SEAL is a desktop examination system for managing practical examinations through a JavaFX client, a Spring Boot REST backend, and a PostgreSQL database. The system supports administrators, teachers, and students. The JavaFX client communicates with the hosted backend over HTTPS, so the application is not limited to a single local area network.

Inclusions:

- User management for creating, viewing, activating, and deactivating teacher and student accounts.
- Examination management, including creating, editing, publishing, and closing examinations.
- Questions with marks, multiple choice options, correct answers, and reference answers for written questions.
- Student exam access using registration number, symbol number, and an exam access code.
- Timed examination attempts with question navigation, multiple choice selection, written answer input, and answer submission.
- Automatic grading for multiple choice questions and manual grading for written answers.
- Storage of users, examinations, questions, options, submissions, answers, marks, and results in PostgreSQL.

Exclusions:

- Webcam monitoring, facial recognition, AI proctoring, and activity monitoring.
- Operating system lockdown, keyboard shortcut blocking, Task Manager restriction, or blocking other applications.
- Automatic saving and recovery of answers while the student is typing.
- Verified university enrollment using registration or symbol numbers.
- Multiple attempts, question randomization, past paper management, and multi institution support.
- Automatic grading of written answers.

The project keeps the name from the proposal. "Lockdown" now means what the software can honestly enforce: the paper is hidden until the start, a paused student cannot type, and the database refuses any change to an answer after the hand-in.

1.5 Report Organization

Chapter 2 reviews the theory behind SEAL and the tools that already exist, and sets out the requirements and the feasibility study. Chapter 3 describes the development model, the timeline and the design: architecture, data flow, database, use cases, the main flowchart and the message sequence. Chapter 4 explains how each module was built and how it was tested. Chapter 5 compares the finished system with each objective and reports measured performance. Chapter 6 closes with the problems we met, the limits of the system and the work that could follow. The references and the annexes, including the user manual, come last.

CHAPTER 2: LITERATURE OVERVIEW

2.1 Background / Theoretical Framework

SEAL combines JavaFX, REST APIs, Spring Boot, and PostgreSQL to manage practical examinations. The JavaFX application is used by administrators, teachers, and students, while the Spring Boot backend handles the main rules and communicates with the database.

The client and server communicate through HTTPS using JSON. Login, exam creation, exam access, submission, grading, and result requests are all handled through REST endpoints. The client does not connect directly to PostgreSQL, which keeps database credentials and application rules on the backend.

SEAL uses role based access control. The user role is stored in the database and returned after a successful login. Administrators manage accounts, teachers manage examinations, and students take published examinations. A session token is created during login and sent with later requests to verify the user's access.

JPA and Hibernate map the backend Java classes to PostgreSQL tables. The database stores users, examinations, questions, question options, submissions, and answers. Relationships and unique constraints help prevent duplicate accounts, exam access codes, attempts, and answers.

The examination process uses different states. Exams can be drafts, published, or closed. Student submissions can be in progress, submitted, or graded. Multiple choice questions are graded automatically by comparing the selected option index with the correct option stored for that question. Written questions are reviewed and graded manually by the teacher.

Passwords are stored using BCrypt hashes rather than plain text. JavaFX background tasks handle network requests so that the user interface remains responsive while the application communicates with the server.

2.2 Study of Related Systems

Table 2.1 summarizes the systems a college could use today to supervise a practical exam.

Table 2.1: Study of Related Systems

S.N.	System	Technology	Key Features	Limitations
1	Safe Exam Browser, ETH Zurich [1]	Kiosk browser, open source	Locks the PC into one quiz page; blocks switching and Task Manager; works with Moodle	Only one web page; the IDE and database client a practical needs are blocked
2	Respondus LockDown Browser [14]	Proprietary browser	Locks an LMS quiz; blocks printing, copying and other programs	Paid licence; online quizzes only; no live view for the invigilator
3	Schoology [15]	Managed secure browser	Central management of secure online assessments	Closed source; same browser-only model; needs the internet
4	Windows kiosk mode (Assigned Access) [3]	Built into Windows	Restricts an account to one program or a fixed set	Set up in advance by an administrator; no exam, clock, answers or reporting
5	Veyon [2]	Classroom management, LAN	Teacher sees and can lock student screens	A teaching tool: no paper, no answer collection, no record of events
6	Manual invigilation	One person walking the rows	Flexible; needs no set-up	Sees a few screens at a time; leaves no record; answers collected by hand

Note: The related systems listed above are cited in IEEE format in the References section.

Each system addresses a different part of examination management. Lockdown browsers restrict the computer but are mainly designed for browser-based assessments. Windows kiosk mode limits access to selected applications but does not manage examination questions, submissions, or grading. Veyon supports classroom monitoring but does not provide an examination workflow or result management. Manual invigilation is flexible but depends heavily on the invigilator and provides limited digital records. SEAL focuses on the examination workflow by providing user management, exam creation, timed student attempts, answer submission, automatic MCQ grading, manual grading of written answers, and result viewing.

2.3 What's New in Our Project?

SEAL addresses the limitations identified in the study of related systems:

- It supports practical examinations through a JavaFX desktop application while allowing students to work with the tools required for their subjects.
- It provides separate workspaces for administrators, teachers, and students through role based authentication.
- It allows teachers to create, edit, publish, and close examinations containing multiple choice and written questions.
- It allows students to access published examinations using an exam access code, complete a timed attempt, navigate between questions, and submit their answers.
- It stores examinations, questions, options, submissions, answers, and marks in a PostgreSQL database through a Spring Boot REST backend.
- It automatically grades multiple choice questions and allows teachers to grade written answers manually.
- It provides teachers with submission and grading views and allows students to view their own examination results.

2.4 Functional and Non-Functional Requirements

The requirements below come from the problem statement and from the review at the proposal defense (see Table 2.2).

Table 2.2: System Requirements

Requirement Type	Description	Priority
Functional - Authentication	The system allows the users to log in and log out using a username and password.	High
Functional – Role Management	The system identifies users as administrators, teachers, or students and provides the appropriate workspace.	High

Requirement Type	Description	Priority
Functional – User Management	An administrator can create, view, activate, and deactivate teacher and students.	High
Functional - Exam Management	A teacher can create, edit, publish, and close examinations.	High
Functional – Questions Management	A teacher can add multiple choice and written questions with marks and related answer information.	High
Functional – Exam Access	A student can add multiple choice and written questions with marks and related answer information.	High
Functional - Grading	The system automatically grades MCQs and allow teachers to grade written answers manually.	High
Functional - Results	Teachers can review submissions and students can view their own submitted and graded results	High
Non-Functional - Security	Passwords are stored using BCrypt, protected requests use bearer tokens, and role permissions are checked by the backend.	High
Non-Functional - Data	Database relationships and unique constraints protect users, exams, submissions, and answers from invalid duplication.	High
Non-Functional - Security	Only SEAL message classes are accepted from the network; SQL uses PreparedStatement.	High

2.5 Feasibility Study

The feasibility study evaluates whether SEAL can be developed and used with the available technology, resources, budget, and project schedule. The current system uses JavaFX for the desktop client, Spring Boot for the REST backend, and PostgreSQL for data storage.

Table 2.3: Feasibility Analysis

Feasibility Type	Analysis	Verdict
Technical	The project uses Java 21, JavaFX, Spring Boot, REST, JPA, Hibernate, PostgreSQL, and Maven. These technologies are suitable for implementing authentication, examination management, submissions, and grading.	Feasible
Operational	Administrators, teachers, and students use separate workspaces. Teachers create exams, students complete timed attempts, and results are managed through the application. The hosted backend requires an internet connection.	Feasible
Economic	The main development tools and libraries are open source. Existing computers can run the JavaFX client, while hosted services can provide the backend and database within the available project budget.	Feasible
Schedule	The work fits into five increments between August and October, as the Gantt chart (Table 3.2) shows.	Feasible
Legal	SEAL records only exam activity on college PCs and tells the student what it records; it takes no screen, camera or key recordings.	Feasible

Based on the above analysis, the project is determined to be fully feasible within the given academic context, available tools, and team capability.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

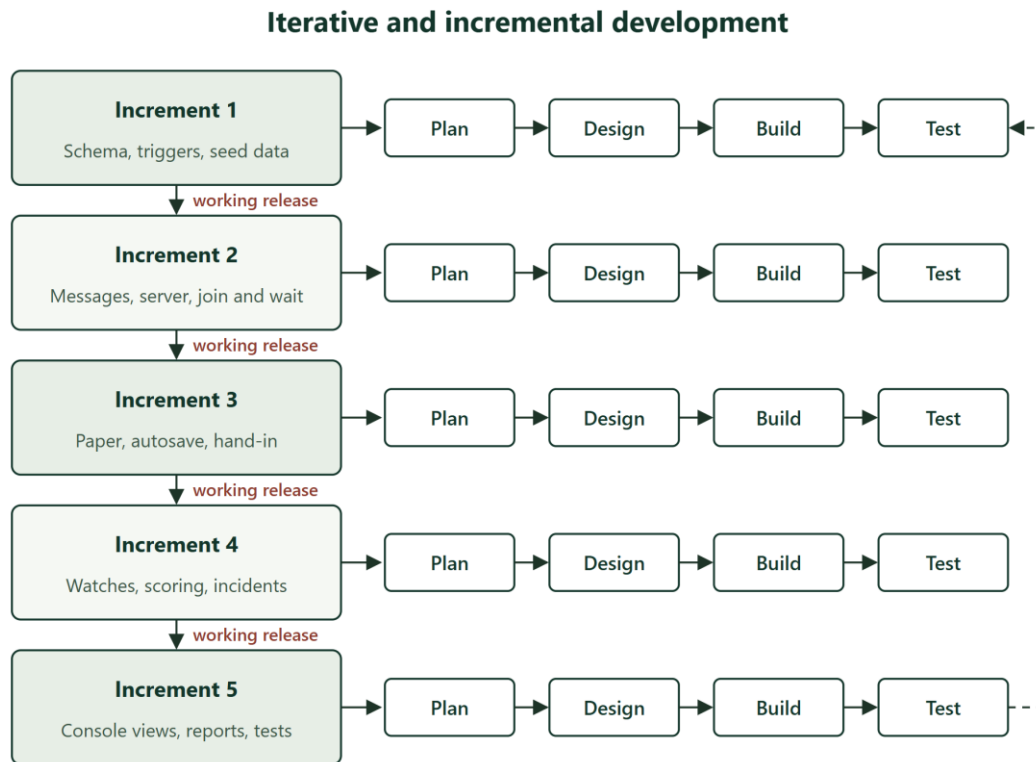
3.1 Software Development Life Cycle (SDLC)

3.1.1 SDLC Model Used

We followed the Iterative and Incremental Development Model [16]. The system was developed in stages, with each stage adding and testing new functionality.

The initial stage implemented the project structure, database connection, authentication, and user sessions. Later stages added role based workspaces, account management, teacher exam creation, student exam access, timed attempts, answer submission, grading, and result viewing.

This model allowed the requirements to be refined during development. Features such as operating system lockdown and screen monitoring were removed from the original proposal, while the core examination workflow was developed and tested incrementally.



Dashed line: what each increment's review found shapes the next one.

Figure 3.1: SDLC Model - Iterative and Incremental

Table 3.1 lists what changed between the proposal and the final system, and why.

Table 3.1: Changes Since the Proposal

Proposal	Final system	Reason
LAN based exam system with a separate invigilator console	JavaFX desktop client with a hosted Spring Boot REST backend	The final system focuses on examination management through a client server architecture.
Operating system lockdown and program restriction	Controlled examination workflow without OS level restrictions	OS level controls require administrator access and were outside the project scope.
Live screen monitoring and event tracking	Exam access, timed attempts, answer submission, grading, and result management	The final system concentrates on the core examination process.
Local autosave and offline answer recovery	Answers are held in the client during the attempt and submitted to the backend	Autosave, backup files, and offline recovery were not implemented.
Invigilator seating and incident reports	Separate administrator, teacher, and student workspaces	The final application provides role based workflows instead of live room supervision.
Automatic monitoring of student computers	Server side validation and grading	The final system validates access, deadlines, submissions, and marks without monitoring other applications.

3.2 Project Timeline - Gantt Chart

The Gantt chart in Table 3.2 shows the timeline from April to October 2026. Shaded cells mark the months in which each activity ran.

Table 3.2: Project Gantt Chart

Activity	Apr	May	Jun	Jul	Aug	Sep	Oct
Requirement study and literature review							
Proposal writing and proposal defense							
Redesign after the defense							
System and database design							
Increments 1 and 2: schema, server, joining							
Increments 3 and 4: answers, watches, scoring							
Increment 5: console views and reports							
Testing and fixes							
Report and final presentation							

Note: Shaded cells show the planned duration of each activity.

3.3 System Architecture

SEAL follows a client server architecture as shown in Figure 3.2. The JavaFX desktop application acts as the client, while the Spring Boot application provides the REST API. The client communicates with the backend through HTTPS using JSON requests and responses.

The Spring Boot backend handles authentication, role based authorization, examination management, submissions, grading, and result retrieval. It connects to the PostgreSQL database through JPA and Hibernate. The database stores users, examinations, questions, options, submissions, answers, and marks.

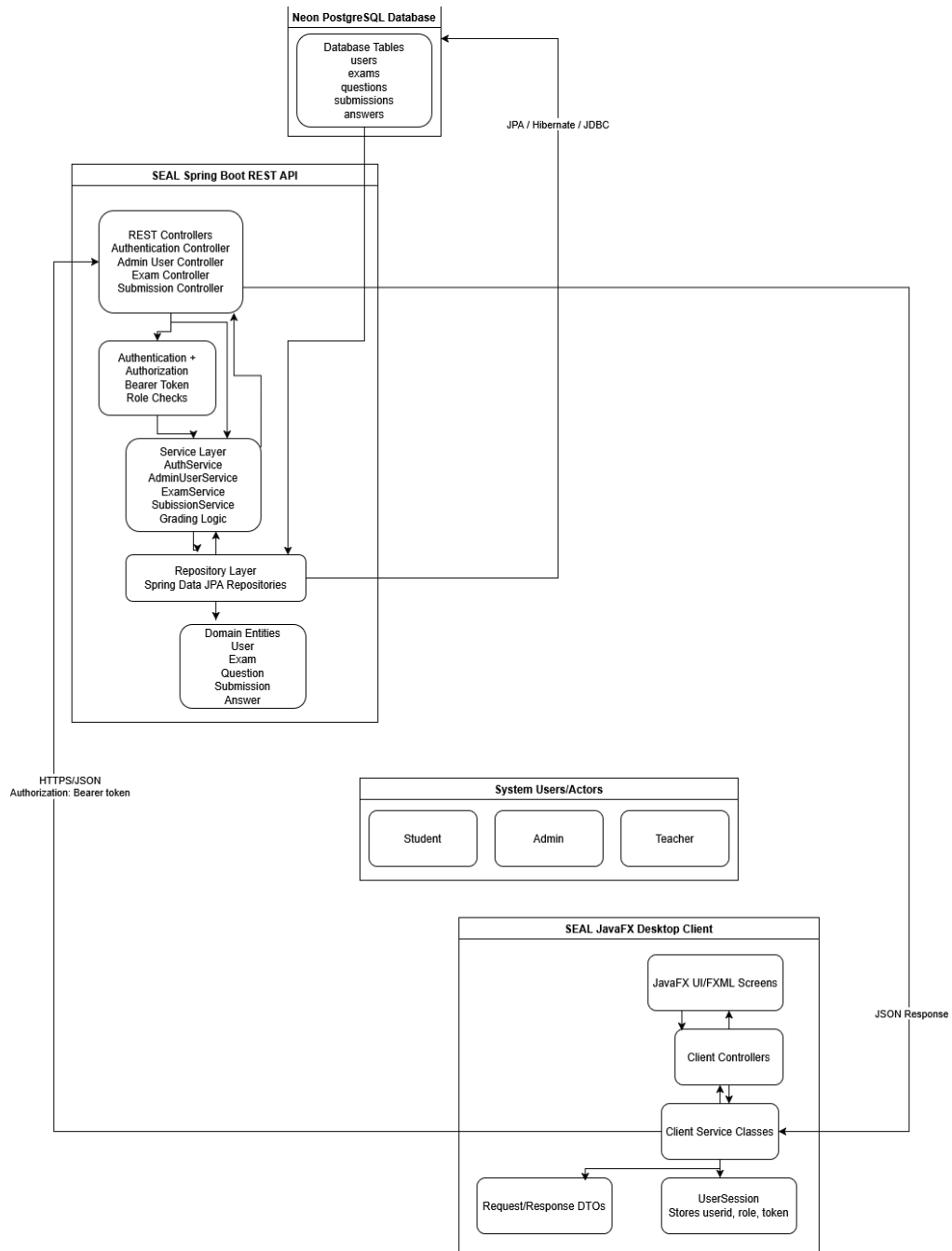


Figure 3.2: System Architecture Diagram

The JavaFX application does not connect directly to PostgreSQL and does not use LAN discovery, UDP broadcasts, or a persistent TCP connection. Each request is sent to the backend endpoint and includes the authenticated user's bearer token when authorization is required.

3.4 Data Flow Diagram (DFD)

The Level 0 DFD shows SEAL as one system connected to three external users: administrator, teacher, and student. Users send login and examination requests, while SEAL returns account information, exam data, submission status, and results.

The Level 1 DFD divides the system into authentication, user management, exam management, student examination, submission, grading, and result processes. These processes communicate with the Spring Boot backend and use PostgreSQL to store users, exams, questions, submissions, answers, and marks.

3.4.1 Level 0 - Context Diagram

At the highest level SEAL exchanges data with three external entities (Figure 3.3). The student sends a roll number, an exam code, answers and, through the watches, observed activity, and receives the paper, the clock, warnings, pauses and a receipt. The invigilator sends the exam set-up, the start and end, warnings and incident decisions, and receives the live room, the events and the incidents. The exam section receives the report, the event log and the answer scripts.

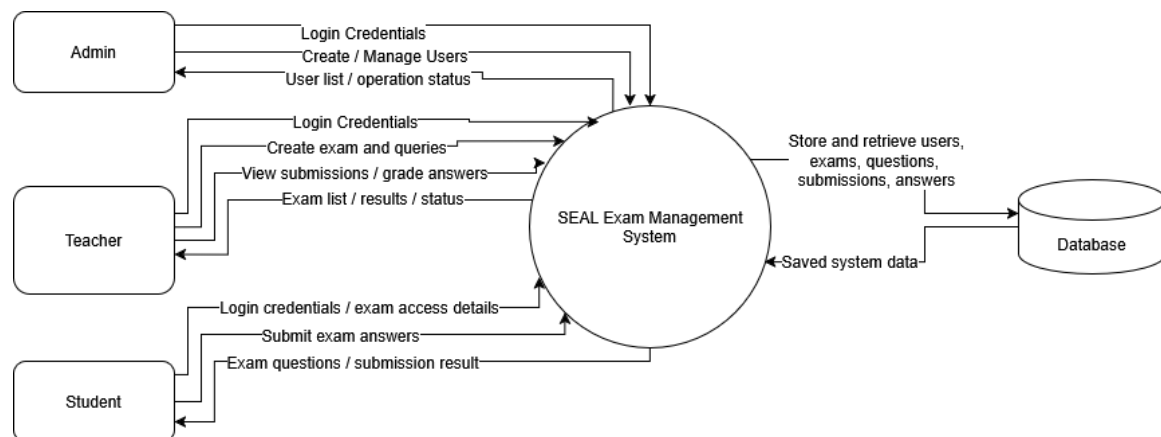


Figure 3.3: Data Flow Diagram - Level 0 (Context Diagram)

3.4.2 Level 1 - Detailed DFD

Figure 3.4 breaks the system into six processes and seven data stores. Joining (1.0) reads the exams and the roster and writes an attempt. Collecting answers (2.0) writes answers, and a trigger writes each revision. Recording activity (3.0) writes events, and the triggers update the attempt and open incidents. The exam lifecycle (4.0) reads and updates the exam and the attempts and logs every command. Reviewing incidents (5.0) reads the

evidence and records the decision. Exporting reports (6.0) reads everything and writes the files for the exam section.

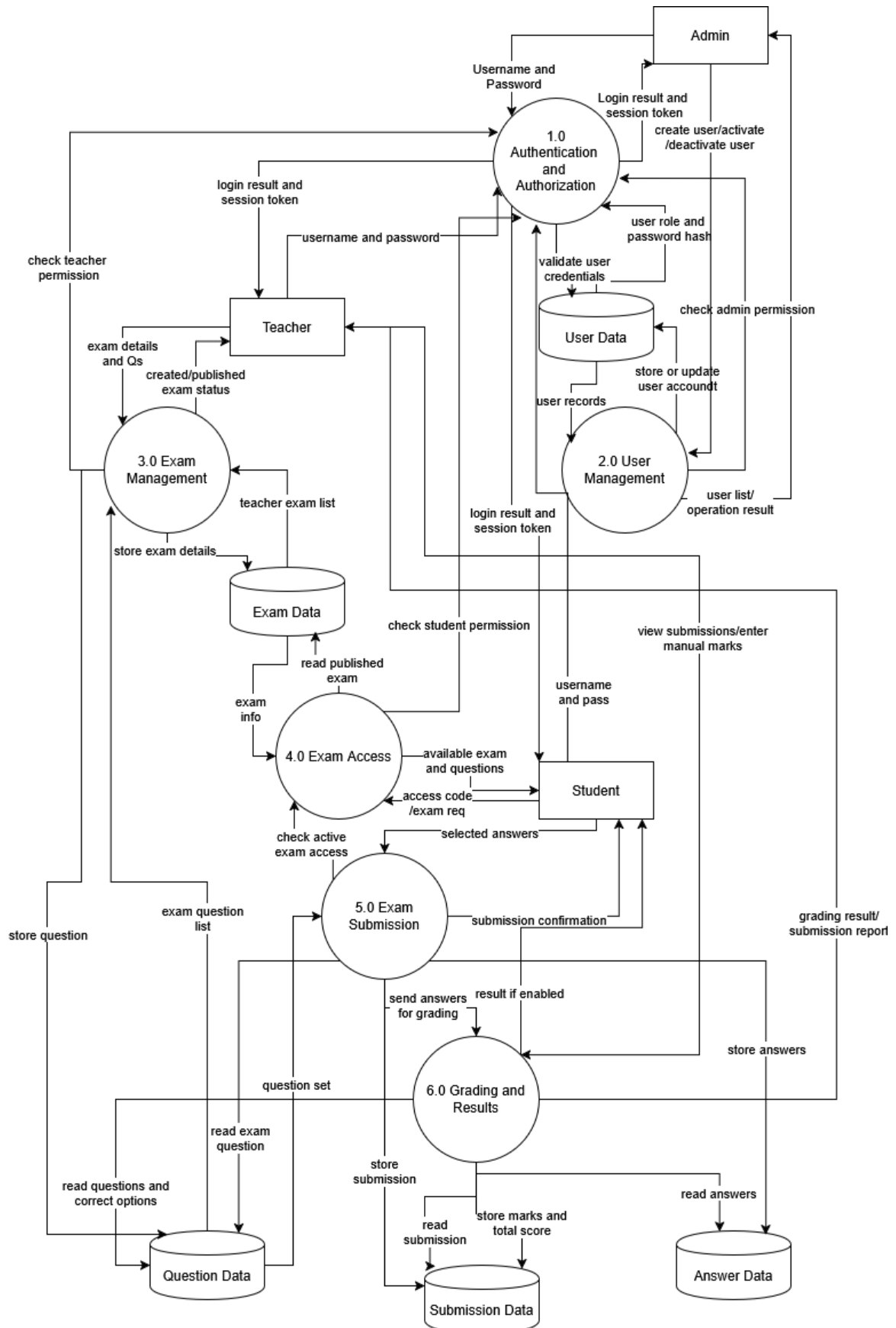


Figure 3.4: Data Flow Diagram - Level 1

3.5 Entity Relationship (ER) Diagram

SEAL uses six main entities: users, exams, questions, question options, submissions, and answers.

A teacher creates exams, and each exam contains questions. MCQ questions contain their available options. Students create submissions for published exams, and each submission belongs to one student and one exam. Answers connect a submission to its questions and store the selected option, written response, and awarded marks.

The database uses primary keys, foreign keys, and unique constraints to maintain these relationships and prevent duplicate submissions or answers.

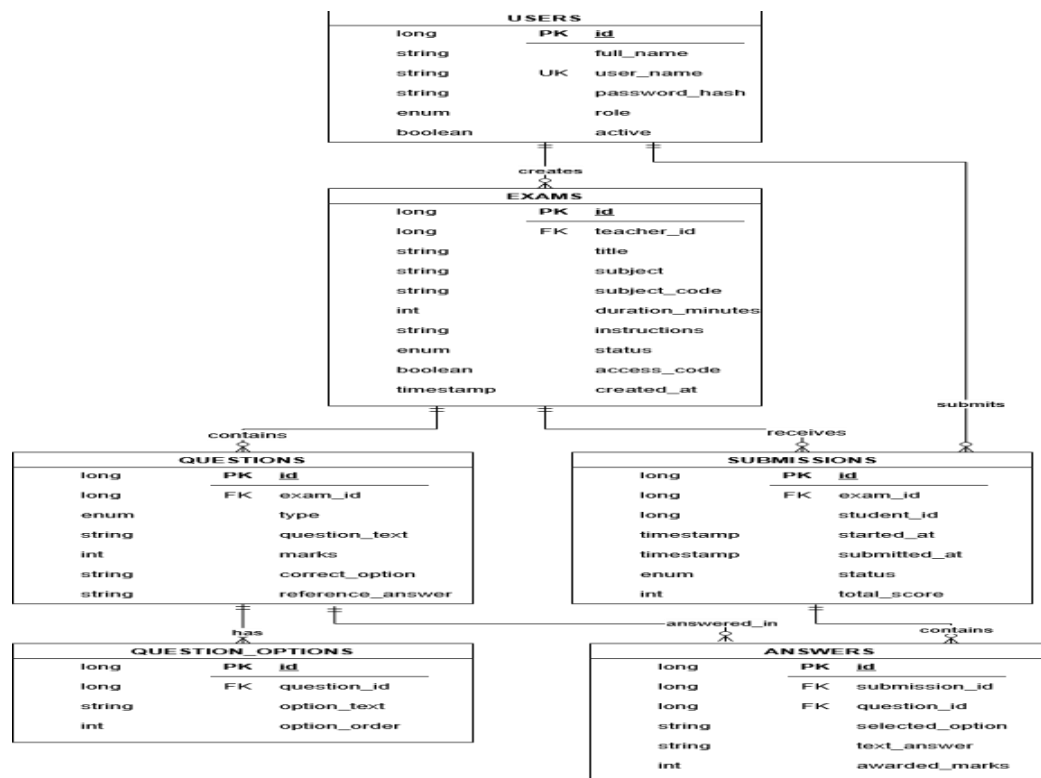


Figure 3.5: Entity Relation Diagram

3.6 Use Case Diagram

As shown in Figure 3.6, SEAL has three main actors: administrator, teacher, and student.

The administrator manages teacher and student accounts. The teacher creates, edits, publishes, closes, and reviews examinations, and manually grades written answers. The student logs in, accesses a published examination using an access code, reads the instructions, starts a timed attempt, answers questions, submits the examination, and views results.

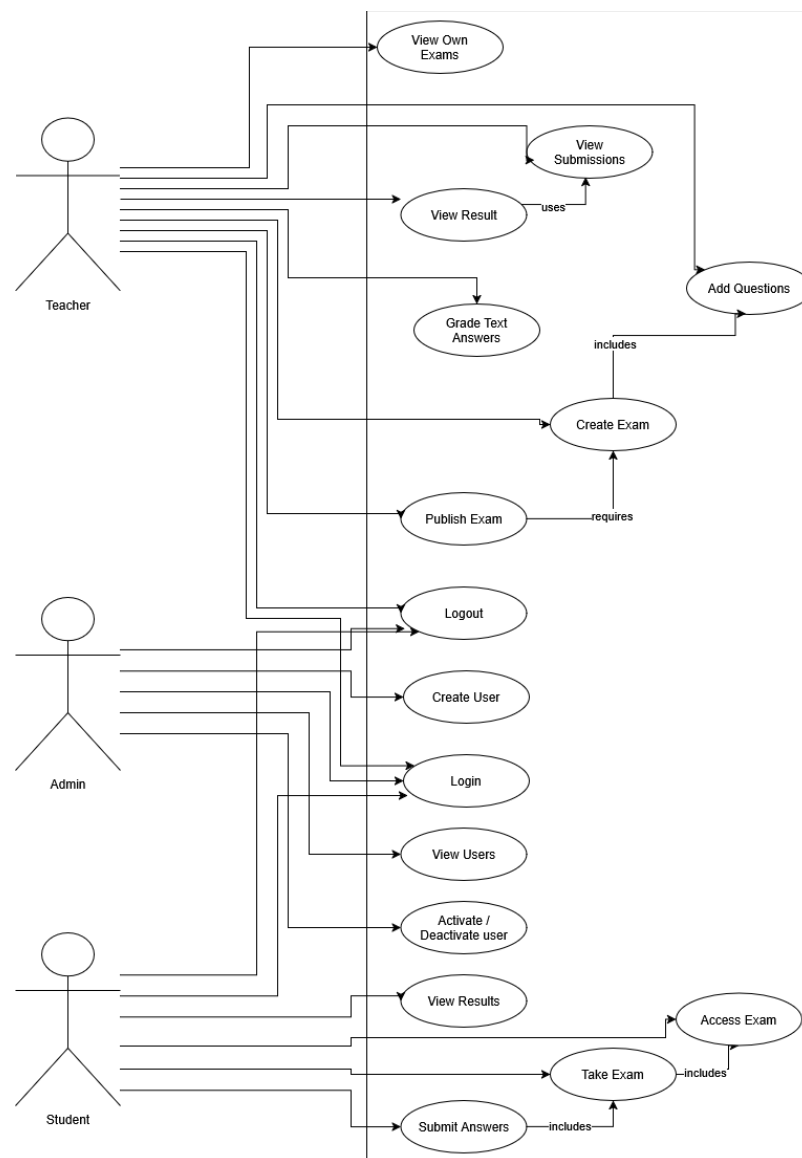


Figure 3.6: Use Case Diagram

3.7 System Flowchart

Figure 3.7 presents the student examination flow. The student logs in, enters the examination details, and accesses a published examination using a valid access code. After reading the instructions, the student starts the timed attempt, answers the questions, and submits the examination. The backend validates the answers, automatically grades MCQs, stores written answers for teacher grading, and displays the submission acknowledgement. The flow ends when the submission is completed or the request is rejected due to invalid details or examination status.

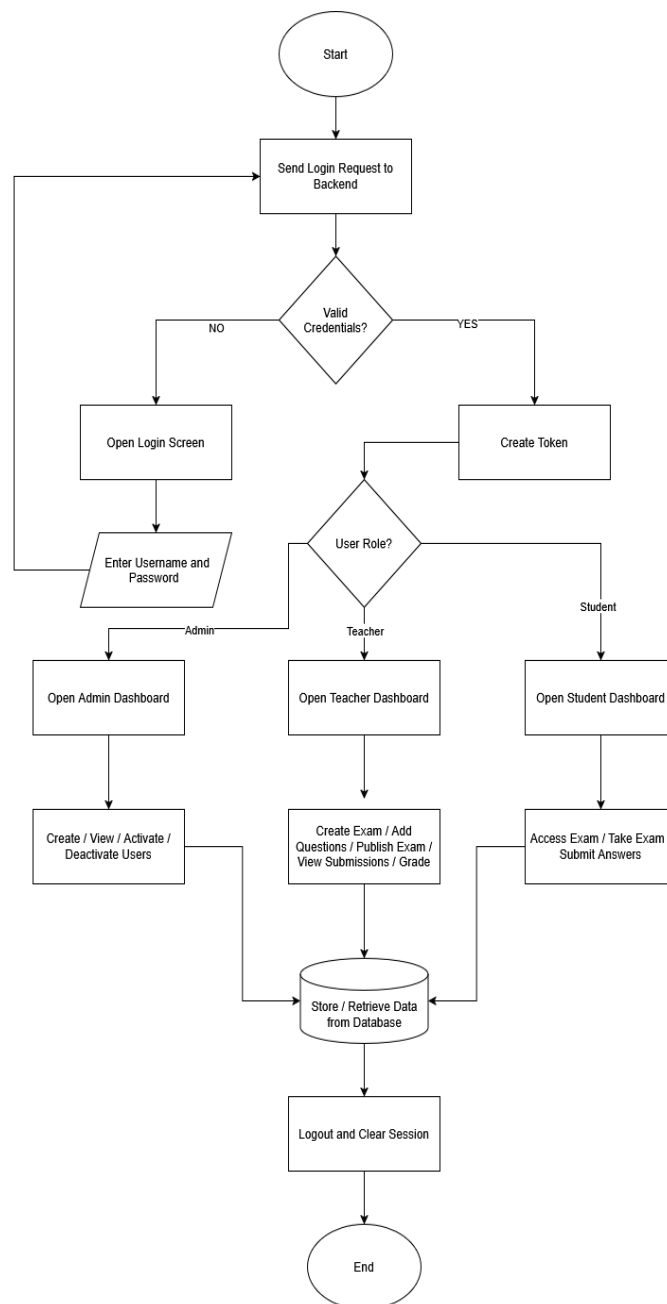


Figure 3.7: System Flowchart

3.8 Message Sequence

Figure 3.8 shows the communication between the JavaFX client, Spring Boot backend, and PostgreSQL database during an examination. The client sends login details, exam access data, start requests, and submitted answers as JSON through HTTPS REST requests. The backend validates the session, role, exam status, and student attempt before responding. During submission, the backend compares MCQ selections with the stored correct options and saves all answers through JPA and Hibernate. The backend returns the submission status and score, while written answers remain available for manual teacher grading.

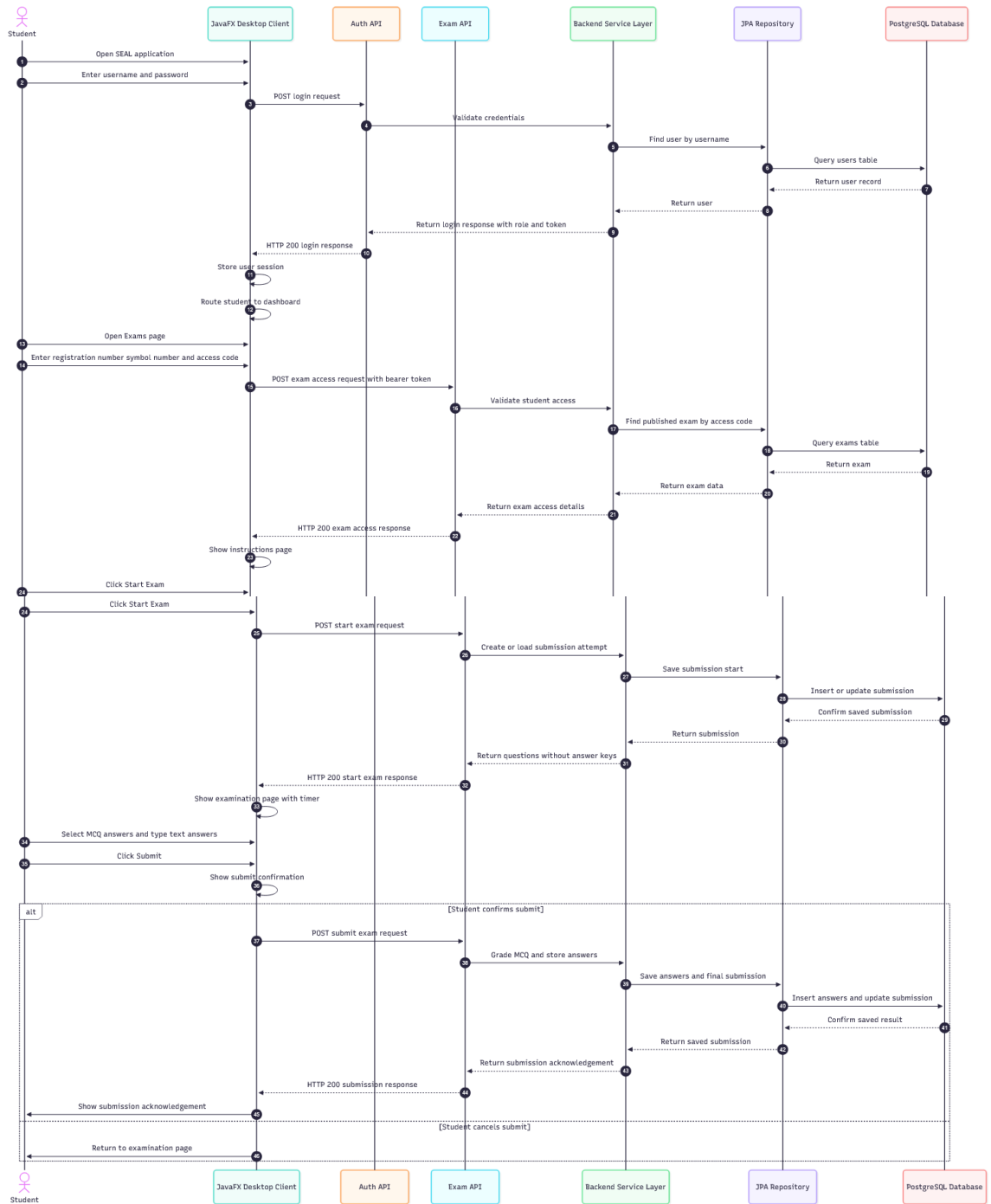


Figure 3.8: Sequence Diagram - Shell, Console and Database

CHAPTER 4: IMPLEMENTATION

4.1 Software and Hardware Requirements

Table 4.1 lists the hardware and software used to develop and run SEAL, with versions.

Table 4.1: Hardware and Software Requirements

Component	Specification / Tool	Purpose
Processor	Intel Core i3 or equivalent (1 GHz+)	Running the console or a shell
RAM	4 GB (8 GB on the invigilator PC)	Console, MySQL and the IDE together
Storage	2 GB free disk space	Program, database and reports
Operating System	Windows 10 / 11	Lab PCs and the invigilator PC
Network	Stable internet connection	Exam traffic and discovery
Language	Java 21	Both applications
User interface	JavaFX 21	Exam shell and console
Database	PostgreSQL	Data, triggers, views, function and procedures
Build tool	Apache Maven	Compiling, tests and packaging
Testing	JUnit 5 with Spring Boot tests	Testing backend functionality
Code Editor / IDE	IntelliJ IDEA / Visual Studio Code	Development environment
Version Control	Git & GitHub	Source code management

4.1.1 Technology Stack and Language Used

The final SEAL system uses the following technologies:

- Language: Java 21. Java is used for both the JavaFX client and the Spring Boot backend.
- Desktop interface: JavaFX 21. FXML defines the layouts, controllers handle user actions, and CSS styles the application screens.
- Backend: Spring Boot. The backend provides REST endpoints for authentication, user management, examinations, submissions, grading, and results.

- Communication: HTTPS and JSON REST. The JavaFX client sends JSON requests to the hosted backend and receives JSON responses. Bearer tokens are used for protected requests.
- Database: PostgreSQL with Neon. PostgreSQL stores users, examinations, questions, options, submissions, answers, and marks.
- Persistence: JPA and Hibernate. These map Java entities to PostgreSQL tables and manage database operations.
- Security: BCrypt and role based authorization. Passwords are stored as BCrypt hashes, while backend roles control access to system functions.
- Build and testing: Maven and JUnit 5. Maven manages compilation, dependencies, testing, and packaging.
- Deployment: Docker and Render. Docker packages the Spring Boot backend, which is deployed through Render.
- Version control: Git and GitHub. Git and GitHub are used for source code management and automated Windows client builds.

Table 4.2 maps the object-oriented and Java concepts of the syllabus to the places where SEAL uses them.

Table 4.2: Syllabus Concepts Used in SEAL

Concept	Where it is used
Inheritance	StudentPage is an abstract base controller reused by student pages such as access, instructions, examination, and acknowledgement.
Polymorphism	Interfaces such as ExamService and PasswordEncoder allow different implementations to be used through the same reference.
Abstraction and interfaces	ExamService, Spring Data repositories, PasswordEncoder, and JavaFX Task provide reusable contracts while hiding implementation details.
Encapsulation	Model fields are private and accessed through methods. Services also hide authentication, validation, grading, and database operations from controllers.
User-defined exceptions	JoinRefusedException carries the reason a join is refused; ExamStateException refuses an action in the wrong exam state.
Records and Data Transfer Objects	Java records such as LoginRequest, LoginResponse, and exam DTOs carry structured data between the client, backend, and REST API.
Multithreading	JavaFX Task objects run network requests in background threads so the user interface remains responsive.
Networking	TCP ServerSocket and Socket; UDP DatagramSocket with broadcast.
Enumerations	UserRole, ExamStatus, QuestionType, and SubmissionStatus restrict values to valid system states.
Generics and Lambda Expressions	Generic collections, JavaFX callbacks, streams, and lambda expressions are used for type safe data processing and event handling.

4.2 Module-wise Implementation

4.2.1 Sign-in and Security Module

The sign in module allows administrators, teachers, and students to access the system using a username and password. The JavaFX login controller sends the credentials to the Spring Boot backend through the REST API. The backend checks the username, compares the password with the stored BCrypt hash, and verifies that the account is active.

After a successful login, the backend creates an opaque session token and returns the user ID, full name, role, and token to the client. The JavaFX application stores this information in UserSession and opens the workspace associated with the user's role.

Protected requests include the token in the Authorization: Bearer header. The backend checks the token, account status, and required role before allowing access.

Password hashes are not returned to the client, and student responses do not include correct answers or teacher marking guides.

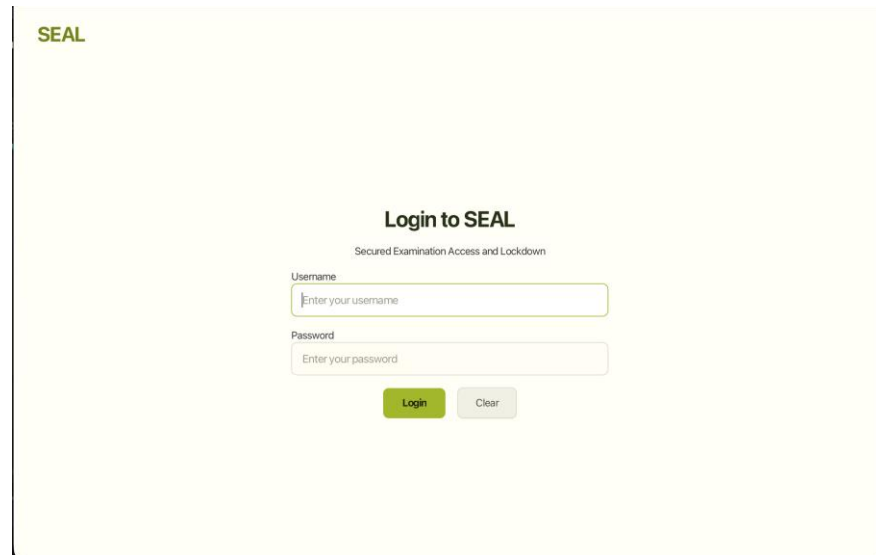


Figure 4.1: Screenshot - Console Sign-in

4.2.2 User and Examination Access Module

Administrators can create, view, activate, and deactivate teacher and student accounts. Teachers can create examinations containing multiple choice and written questions. An examination can be saved as a draft, published with a generated access code, and later closed.

Students first log in and open the Exams section from the student dashboard. They enter their registration number, symbol number, and examination access code. The backend checks the student's session, verifies that the examination exists and is published, and creates or retrieves the student's examination submission.

The current system does not use LAN discovery, UDP broadcasts, or a separate invigilator console. Registration and symbol numbers are stored as submitted examination information and are not checked against a separate enrollment database.

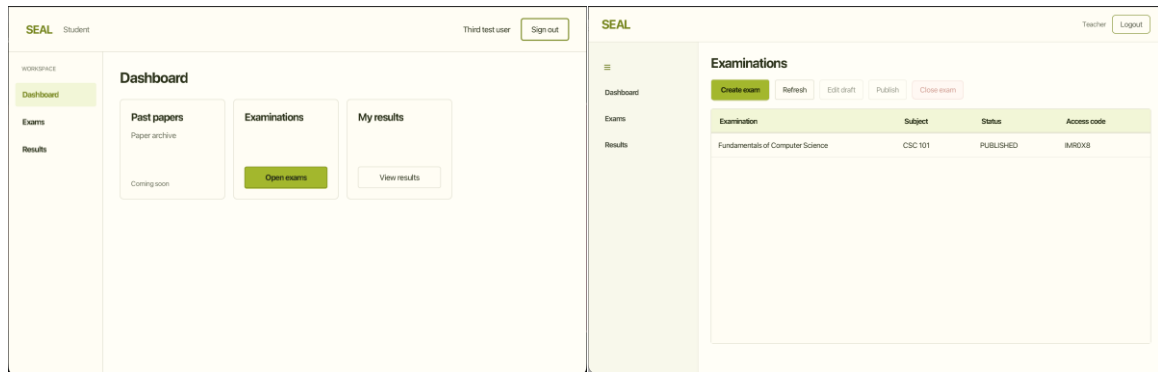


Figure 4.2: Screenshot – Two dashboard student and teacher

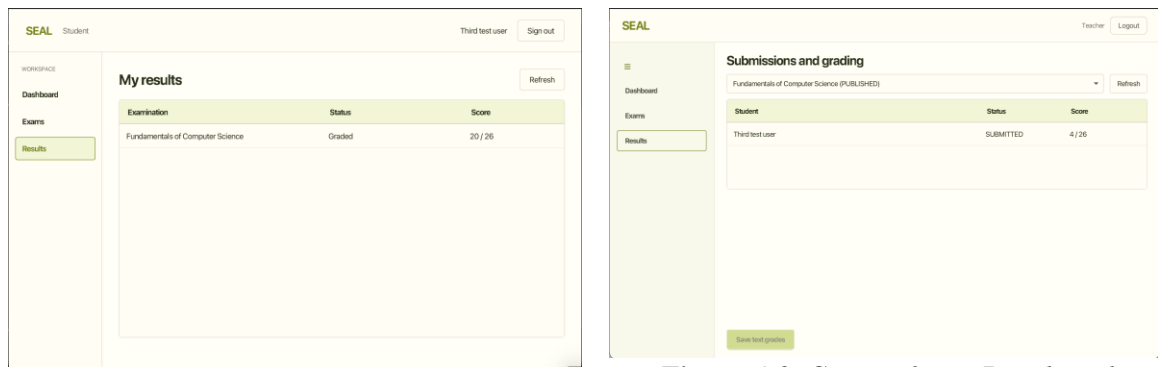


Figure 4.3: Screenshot – Result and Submission panel

4.2.3 Examination and Answer Submission Module

After access validation, the student views the examination instructions and accepts them before starting. The backend records the start time and calculates the deadline using the examination duration.

The JavaFX examination page displays questions, marks, navigation controls, MCQ options, written answer fields, and a countdown timer. Student answers are temporarily held in the client while the examination is in progress. They are submitted to the backend when the student confirms submission or when the client timer reaches zero.

The backend validates that each question belongs to the examination and that the answer type is correct. It then stores one answer for each question in the database. Multiple choice answers are graded automatically, while written answers remain available for manual teacher grading. The system does not currently implement autosave, local answer backups, or offline answer recovery.

The screenshot shows a web interface for a student. At the top, it says 'SEAL Student' on the left and 'Third test user' with a 'Sign out' button on the right. A sidebar on the left is labeled 'WORKSPACE' and contains three items: 'Dashboard', 'Exams' (which is highlighted with a green border), and 'Results'. The main content area is titled 'Exams' and contains a 'Student examination' section. Below this is an 'Exam access' section with three input fields: 'Registration number', 'Symbol number', and 'Exam access code'. At the bottom of this section is a green 'Continue' button.

Figure 4.4: Screenshot – Student registering for exam

4.2.4 Grading and Results Module

For each MCQ, the backend compares the student's selected option index with the correct option index stored for the question. A correct answer receives the question's full marks, while an incorrect or unanswered question receives zero.

If the examination contains written questions, the submission initially remains in a submitted state while the teacher reviews those answers. The teacher enters marks within the maximum allowed for each question. The backend then recalculates the total score and changes the submission status to graded.

Teachers can view submissions and grading details for their own examinations. Students can view summaries of their own submitted and graded examinations. Live screen monitoring, event tracking, warnings, process monitoring, and incident reporting are outside the current implementation.

4.2.5 Teacher Results and Grading Module

The teacher results page displays examinations created by the logged in teacher and lists the submissions received for the selected examination. The teacher can select a submission to view the student's answers, question marks, and the reference answers for written questions.

MCQ marks are already calculated by the backend. For written questions, the teacher enters the awarded marks using the grading controls. The backend checks that the marks are within the question's maximum and then recalculates the total score. Teachers can only view and grade submissions belonging to their own examinations.

4.2.6 Student Results and Acknowledgement Module

After submitting an examination, the student receives an acknowledgement containing the submission reference, status, and current score. If written questions are included, the acknowledgement indicates that manual grading is still pending.

The student results page retrieves only the authenticated student's submitted and graded examinations. It displays the examination title, submission status, and score. Students cannot view another student's submissions, teacher marking guides, or correct answer information.

4.2.7 Persistence and Deployment Module

SEAL stores its permanent data in PostgreSQL. JPA and Hibernate map the backend entities to the database tables and manage users, examinations, questions, options, submissions, answers, and marks. Transactions and database constraints help maintain consistent records.

The JavaFX client communicates with the Spring Boot backend through HTTPS and JSON REST requests. The backend is packaged with Docker and deployed on Render, while the PostgreSQL database is hosted on Neon. The desktop client can be packaged with its Java runtime for Windows distribution.

Live room monitoring, risk scoring, incident management, automatic offline collection, answer history, and exported event reports are outside the current implementation.

4.3 Testing

Testing was carried out on the main authentication, user management, examination, submission, grading, and result workflows, Backend integration tests use a temporary H2 database so that they do not modify the production PostgreSQL database.

Test Area	Expected Result
Login	Correct credentials succeed, while incorrect or inactive accounts are rejected.
Role authorization	Users can access only the functions allowed to their role.
User management	Administrators can create, list, activate, deactivate accounts.
Exam Management	Teachers can create, edit, publish, and close their own examination.
Student attempt	Student can validate access, start a timed exam, and submit answers.
MCQ grading	Correct answers receive marks, while incorrect and unanswered questions receive zero.
Written grading	Teachers can assign marks within the question's maximum.
Result isolation	Students can view only their own submitted and graded results.

Table 4.3: Test Cases

Note: All test cases were run on the final build. No test case failed in the final run; the failures found during development are described in Section 4.3.3.

4.3.3 Test Results Summary

The tests confirmed the main SEAL workflows, including authentication, role based access, examination management, student submissions, automatic MCQ grading, manual written answer grading, and result protection. Invalid requests, duplicate answers, unauthorized access, inactive accounts, and invalid examination states were also tested. Features such as live monitoring, incident management, autosave, offline recovery, and event reporting were not tested because they are outside the current implementation.

CHAPTER 5: FINAL OUTCOME

5.1 System Output and Analysis

The final system consists of a JavaFX desktop application connected to a Spring Boot REST backend and PostgreSQL database. Users sign in according to their roles: administrator, teacher, or student.

Administrators manage user accounts. Teachers create and publish exams, add questions, review submissions, assign marks for written answers, and view results. Students access published exams using the required details, complete the timed examination, submit their answers, and view the available result information.

MCQ answers are checked automatically by comparing the selected option with the correct option stored for the question. Text answers remain available for manual marking by the teacher. The system stores submissions, answer records, marks, exam status, and user information in the database.

The final implementation does not include live computer monitoring, incident scoring, automatic answer collection after disconnection, LAN discovery, or operating-system lockdown. These features remain outside the current scope.

5.2 Analysis of Output Against Objectives

Objective 1:

To design a secure examination system with role-based access.
Outcome: SEAL uses a JavaFX client, Spring Boot REST backend, and PostgreSQL database. Authentication and role checks separate the administrator, teacher, and student functions.

Objective 2:

To implement the examination management and student attempt workflow.
Outcome: Teachers can create and publish exams with MCQ and written questions. Students can access published exams, complete timed attempts, submit answers, and receive a submission acknowledgement.

Objective 3:

To support grading and result management. Outcome: MCQ answers are graded automatically by comparing the selected option with the stored correct option. Written answers remain available for teacher marking, while submissions and results are stored and displayed according to the user's role.

Testing confirmed the main authentication, account management, examination, submission, grading, and result workflows. Features such as live monitoring, LAN discovery, autosave recovery, incident scoring, and operating-system lockdown were not included in the final system.

5.3 Final System Screenshots

The screenshot shows the SEAL Admin panel's User Management section. The top bar includes the 'SEAL' logo and 'Administrator Logout' links. The sidebar on the left has a menu icon, 'Users', and '+Add user'. The main content area is titled 'User Management' with the subtitle 'Manage Teacher and Student accounts'. It features 'Refresh' and 'Deactivate' buttons above a table of users.

Name	Username	Role	Status
Kabin Bagale	Kabin	STUDENT	Active
Test Teacher	teacher1	TEACHER	Active
First test user	test1_teacher	TEACHER	Active
second test user	test_user2_teacher	TEACHER	Active
Third test user	test_student1	STUDENT	Active
test_student2	test_student2	STUDENT	Active

Figure 5.1: Screenshot – Admin panel demonstrating active and passive users

5.4 Performance and Usability Observations

The main workflows were tested on the development system, including login, account management, exam creation, student access, timed answering, submission, grading, and result viewing.

The JavaFX interface provides separate workspaces for each role. Teachers can manage exams and review submissions, while students can move through the examination using clear navigation controls. Backend operations run through REST requests, and validation messages are shown when login details, access codes, exam states, or submitted answers are invalid.

The application performed reliably during functional testing. No live-room monitoring, offline recovery, or multi-computer LAN performance measurements were carried out because these features are outside the final implementation. Further testing with more users and a larger deployed database would be required before production use.

CHAPTER 6: CONCLUSION, LIMITATIONS & FUTURE RECOMMENDATIONS

6.1 Conclusion

SEAL was developed as a role-based examination system using a JavaFX client, Spring Boot REST backend, and PostgreSQL database. It supports administrator account management, teacher exam creation, student exam access, timed answering, submission, MCQ auto-grading, manual written-answer grading, and result viewing.

The system achieved the main objectives by providing a complete examination workflow with authentication, validation, persistent storage, and separate functions for each user role. The final implementation focuses on managing examinations and results rather than enforcing operating-system lockdown or monitoring student computers.

6.2 Problems Faced and Limitations

During development, the main challenges involved connecting the JavaFX client with the REST backend, validating different user roles, managing exam states, and handling both MCQ and written answers.

Current Limitations:

- The system does not prevent students from using phones or other devices.
- It depends on a working internet connection to reach the deployed backend.
- Text answers require manual grading by the teacher.
- The system has not been tested with a large real examination class.
- Advanced monitoring, offline recovery, webcam use, and operating-system lockdown are not included.

6.3 Future Recommendations

- Adding past-paper access and improved student progress tracking.
- Providing richer reports and analytics for teachers.
- Introducing institutional deployment with stronger security and scalability.
- Testing the system with a larger number of real users.

REFERENCES

- [1] Educational Technologies, ETH Zurich, “Safe Exam Browser,” SafeExamBrowser.org. [Online]. Available: <https://safeexambrowser.org>. [Accessed: 8 Oct. 2026].
- [2] Veyon Solutions, “Veyon: Virtual Eye On Networks,” Veyon. [Online]. Available: <https://veyon.io>. [Accessed: 8 Oct. 2026].
- [3] Microsoft, “Set up a single-app kiosk on Windows,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/windows/configuration/kiosk-single-app>. [Accessed: 8 Oct. 2026].
- [4] Oracle Corporation, “Java Platform, Standard Edition 21 API Specification,” Oracle. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/docs/api/>. [Accessed: 8 Oct. 2026].
- [5] OpenJFX, “JavaFX 21 API Documentation,” OpenJFX. [Online]. Available: <https://openjfx.io/javadoc/21/>. [Accessed: 8 Oct. 2026].
- [6] Spring, “Spring Boot Reference Documentation,” Spring. [Online]. Available: <https://docs.spring.io/spring-boot/reference/>. [Accessed: 8 Oct. 2026].
- [7] Spring, “Spring Framework Reference Documentation,” Spring. [Online]. Available: <https://docs.spring.io/spring-framework/reference/>. [Accessed: 8 Oct. 2026].
- [8] Hibernate, “Hibernate ORM Documentation,” Hibernate. [Online]. Available: <https://hibernate.org/orm/documentation/>. [Accessed: 8 Oct. 2026].
- [9] Jakarta EE, “Jakarta Persistence Specification,” Jakarta EE. [Online]. Available: <https://jakarta.ee/specifications/persistence/>. [Accessed: 8 Oct. 2026].
- [10] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL. [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed: 8 Oct. 2026].
- [11] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: 8 Oct. 2026].
- [12] Apache Maven Project, “Maven Documentation,” Apache Maven. [Online]. Available: <https://maven.apache.org/guides/>. [Accessed: 8 Oct. 2026].
- [13] JUnit Team, “JUnit 5 User Guide,” JUnit. [Online]. Available: <https://docs.junit.org/5>. [Accessed: 8 Oct. 2026].
- [14] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.
- [15] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

ANNEX

Annex A: Source Code

The complete source code is available in the project repositories:

GitHub Repository: https://github.com/KabinThapa/SEAL_education_lockdown

The repositories contain the JavaFX interface, FXML files, CSS stylesheets, controllers, REST client classes, Spring Boot controllers, services, repositories, entities, configuration files, and tests.

Annex B: User Manual

SEAL requires Java 21 or newer on the desktop client. The backend is deployed separately as a Spring Boot service connected to PostgreSQL.

Installation

- Download the appropriate desktop build for the operating system.
- Extract the application folder.
- Start the SEAL application using the provided launcher.
- Ensure the computer has an internet connection so it can reach the deployed backend.
- Sign in using the account details provided by the administrator.

Administrator

- Create user accounts
- View users
- Activate or deactivate accounts
- Manage user roles

Teacher

- Sign in.
- Open the teacher dashboard.
- Create an examination.
- Add MCQ or written questions.
- Set marks and correct MCQ options.
- Save the exam as a draft or publish it.

- View student submissions.
- Grade written answers.
- Review and publish results.

Student

- Sign in.
- Open the student dashboard.
- Select the examination area.
- Enter the registration number, symbol number, and exam access code.
- Read the instructions.
- Start the timed examination.
- Answer MCQ and written questions.
- Submit the examination.
- View the submission acknowledgement and available results.